Vision Meets Language

Building a Multimodal Transformer from Scratch

Seasons of Code · IIT Bombay

Mentee Handbook · Tasks 5 and 6

Contrastive Learning and Training on Real Paired Data

Preface — Where We Are, and What This Pair of Tasks Is For

Tasks 0 through 4 built the architectural skeleton of a Vision-Language Model. You have a text transformer, a Vision Transformer, and the cross-attention machinery that connects them. What you do not yet have is anything resembling intelligence. The model has the shape of one that could match images to text, but it has never been given a reason to actually learn that matching.

Tasks 5 and 6 are where the project starts learning. Task 5 defines the loss function — InfoNCE, the contrastive objective behind CLIP and most modern vision-language alignment models. Task 6 trains the full model on real paired image-caption data (Flickr8k) and produces the first attention maps that actually mean something — where the model attends to the dog when the caption says "dog", and to the sky when it says "sky".

A new feature in this handbook: Applied Stretch tasks. These are short, real-world applications of the concepts you've just learned — captioning, search, retrieval, classification without labels — that demonstrate how the same machinery powers products you've actually seen. They are completely optional and live at the end of each task. They exist for mentees who finish core work early and want to feel what it's like to apply these ideas to something concrete. They also serve a longer-term purpose: when someone asks you in an interview "have you ever applied contrastive learning to anything?", you want to be able to say yes.

Hardware note. Task 6 needs a GPU. Free Google Colab (T4) is sufficient for our scale. If you have a local GPU, use it — turnaround time will be much faster. CPU training is technically possible but will take days; do not attempt it.

Task 5 — Contrastive Learning and InfoNCE Loss

Goal: Understand contrastive learning as a paradigm, derive InfoNCE loss from first principles, implement it carefully (with all the numerical-stability tricks intact), and verify it produces meaningful gradients on toy data before you let it loose on real images and captions in Task 6.

Why this task exists

Up to now, every loss function you have used has been classification cross-entropy — given an input and a fixed set of labels, predict the right label. Contrastive learning is structurally different. There are no fixed labels. The model learns by being shown pairs that should be similar and being told "these are similar; everything else in your batch is dissimilar." The mathematics of this looks like cross-entropy, but the semantics are completely different. Understanding this difference precisely is the goal of Task 5.

The Contrastive Idea, From Scratch

Start with a clean mental model. You have a batch of `N` image-caption pairs, where pair `i` is `(image_i, caption_i)`. For each image, *its own* caption is the positive (the matching pair), and all `N-1` other captions in the batch are negatives (random captions that happen to be in the same batch). The model's job: produce image and text embeddings such that `image_i` is closer to `caption_i` than to any other caption in the batch.

The setup geometrically. Both the image encoder and text encoder produce embeddings in the same D-dimensional space (this is why we standardized to embed_dim=192 in Task 4). After L2-normalization, every embedding lies on a unit sphere. Similarity is measured by cosine similarity, which is just the dot product on the unit sphere.

The InfoNCE loss for one direction. For each image, treat similarities to all N captions as logits, the correct caption's index as the target label, and apply cross-entropy:

```
# pseudocode for one direction (image -> text)
# I: (N, D) image embeddings, L2-normalized
# T: (N, D) text embeddings, L2-normalized
# tau: scalar temperature

logits = I @ T.T / tau           # (N, N) similarity matrix
labels = torch.arange(N)         # diagonal: pair i matches caption i
loss_i2t = cross_entropy(logits, labels)
```

The full InfoNCE loss is symmetric: compute the loss with images as queries and texts as keys, then again with texts as queries and images as keys, and average. This is the CLIP loss.

```
loss = (loss_i2t + loss_t2i) / 2
```

Derivation exercise. On paper, in `task5/derivation.pdf`:

1. Write down the InfoNCE loss for one direction (image to text) as an explicit summation, in terms of the cosine similarities s_{ij} and temperature tau. Don't just state it — derive it from the principle of "maximize log probability of the correct pair under a softmax over candidates."
2. Show that as tau → 0, the loss becomes "correct pair must be strictly greater than all others" (max-margin-like behavior). Show that as tau → infinity, the loss collapses to a constant. Explain why.
3. Compute ∂loss_i2t / ∂s_{ii} (the gradient with respect to the correct pair's similarity) and ∂loss_i2t / ∂s_{ij} for j ≠ i (an incorrect pair). Interpret the signs: which gradients pull pairs together and which push them apart?

CURIOSITY CORNER — Why this is called "InfoNCE"

InfoNCE = "Information Noise Contrastive Estimation". The name comes from a 2018 paper by van den Oord et al. They showed that the loss is a lower bound on the mutual information between the two views being contrasted. In our setting: maximizing InfoNCE between images and their captions is equivalent (in a precise mathematical sense) to maximizing the mutual information between image content and caption content.

This connection to information theory is a deep reason the loss works so well across so many domains — vision-language, video-audio, sentence pairs, protein sequences. Anywhere you have two related views of the same thing, you can contrastively align them.

CURIOSITY CORNER — The temperature tau is doing a lot of work

Temperature is the single most impact-per-line hyperparameter in CLIP-style training. Too high (~1.0): logits are flat, gradients are weak, training is slow. Too low (~0.01): logits are sharp, the model becomes obsessed with hard negatives, training collapses or gets unstable.

CLIP famously learns temperature as a parameter, initialized to log(1/0.07). This means tau starts at ~0.07, which is much sharper than uniform but not extreme. You'll do this too.

CURIOSITY CORNER — Contrastive learning has a 30-year history

Although CLIP made contrastive learning famous in 2021, the idea is much older. Siamese networks (Bromley et al., 1993) trained twin networks to produce similar outputs for matching pairs of signatures. Triplet loss (FaceNet, 2015) extended this with explicit "positive vs negative" triplets — but required mining hard negatives manually.

InfoNCE's contribution was treating all other examples in the batch as negatives "for free", which made the method massively scalable. SimCLR (2020) demonstrated this for self-supervised image

learning. CLIP (2021) extended it across modalities. Today the same loss powers DINO, MoCo, ALIGN, BLIP, and dozens of others.

Implementing InfoNCE Carefully

Build the loss in `task5/loss.py`:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class InfoNCELoss(nn.Module):
    def __init__(self, init_temperature=0.07):
        super().__init__()
        # Learnable log-temperature, as in CLIP
        # log(1/0.07) ~ 2.659
        self.log_inv_tau = nn.Parameter(
            torch.tensor([1.0 / init_temperature]).log()
        )

    def forward(self, image_embeds, text_embeds):
        # Normalize to unit sphere
        image_embeds = F.normalize(image_embeds, dim=-1)
        text_embeds  = F.normalize(text_embeds,  dim=-1)

        # Clamp temperature for stability (CLIP does this)
        # Prevents log_inv_tau from blowing up
        log_inv_tau = self.log_inv_tau.clamp(0, 4.6052)  # log(100)
        inv_tau = log_inv_tau.exp()

        # Similarity matrix (N, N)
        logits = inv_tau * image_embeds @ text_embeds.T

        # Targets are the diagonal
        N = image_embeds.size(0)
        labels = torch.arange(N, device=logits.device)

        loss_i2t = F.cross_entropy(logits, labels)
        loss_t2i = F.cross_entropy(logits.T, labels)
        return (loss_i2t + loss_t2i) / 2
```

⚠ Three things people get wrong on first implementation

1. Forgetting to L2-normalize before the dot product. Without normalization, similarity is dominated by embedding magnitudes, not direction. The model can lower the loss by making embeddings shorter or longer — completely bypassing the alignment objective.

2. Using a fixed temperature. Fixed temperature works, but learned temperature consistently helps. Crucially, parameterize it as log(1/tau) — gradient descent on log-temperature is much more stable than on temperature directly.

3. Not clamping log-temperature. Without clamping, training can drive temperature to extreme values and destroy gradients. CLIP clamps it. You should too.

Sanity tests. Verify in `task5/test_loss.py`:

4. Pass identical `image_embeds == text_embeds` (same random tensor for both). The loss should be very low — they are already perfectly aligned. What value do you expect, and why?
5. Pass `image_embeds = torch.randn(N, D)` and `text_embeds = torch.randn(N, D)` (independent random). The loss should be approximately `log(N)`. Why?
6. Pass a controlled mixture: half the pairs aligned (same vector for image and text), half random. Loss should be somewhere between the two above. Verify.
7. Call backward() and confirm gradients exist on log_inv_tau and on both input embeddings.

CURIOSITY CORNER — Why does random-embedding loss equal log(N)?

If image and text embeddings are uncorrelated, the softmax over N candidates puts roughly equal probability 1/N on the correct one. The cross-entropy loss is then -log(1/N) = log(N). For a batch of 128, this is about 4.85.

This is your sanity baseline. At step 0 of training, with a freshly initialized model, your loss should be very close to log(N). If it's not, something is wrong with normalization, temperature, or the batch construction.

Building the Full Training Forward Pass

Connect your Task 4 VLM to the InfoNCE loss. The two need a small interface change: the loss expects single-vector embeddings per image and per caption, but your encoders produce sequences. You need a pooling step.

The full training architecture:

8. Image (B, 3, H, W) → ViT encoder → `(B, num_patches+1, D)` → take CLS token → `(B, D)` → image projection head → `(B, D_proj)` → L2 normalize.
9. Text (B, T_text) → text encoder → `(B, T_text, D)` → take last token or mean-pool → `(B, D)` → text projection head → `(B, D_proj)` → L2 normalize.
10. InfoNCE loss between the two.

Important: drop the cross-attention from your Task 4 VLM. For contrastive training, the two encoders run independently and only meet at the loss. This is the CLIP-style architecture. Cross-attention as we built it in Task 4 is used for *conditional* tasks (captioning, VQA). For *retrieval-style alignment* tasks, two separate encoders with contrastive loss are simpler and more effective.

Build `task5/clip_model.py`:

```
class CLIPStyleModel(nn.Module):
    def __init__(self, vit_encoder, text_encoder,
                 embed_dim=192, projection_dim=128):
        super().__init__()
        self.vit  = vit_encoder
        self.text = text_encoder
        self.image_proj = nn.Linear(embed_dim, projection_dim, bias=False)
        self.text_proj  = nn.Linear(embed_dim, projection_dim, bias=False)
        self.loss_fn = InfoNCELoss(init_temperature=0.07)

    def encode_image(self, images):
        feats = self.vit.encode(images)                 # (B, N+1, D)
        cls   = feats[:, 0]                             # (B, D)
        return self.image_proj(cls)                     # (B, D_proj)

    def encode_text(self, text_tokens, text_mask=None):
        feats = self.text.encode(text_tokens)           # (B, T, D)
        # Mean-pool ignoring padding
        if text_mask is not None:
            mask = text_mask.unsqueeze(-1).float()      # (B, T, 1)
            pooled = (feats * mask).sum(1) / mask.sum(1).clamp(min=1e-6)
        else:
            pooled = feats.mean(1)
        return self.text_proj(pooled)                   # (B, D_proj)

    def forward(self, images, text_tokens, text_mask=None):
        img_e  = self.encode_image(images)
        txt_e  = self.encode_text(text_tokens, text_mask)
        loss   = self.loss_fn(img_e, txt_e)
        return loss, img_e, txt_e
```

Verify with dummy data. Same drill as Task 4 Day 3: random images + random text tokens, forward, backward, check gradients flow everywhere. Verify the loss is approximately `log(batch_size)` at initialization.

> **CURIOSITY CORNER — Mean-pool vs CLS vs last-token for text**
>
> Three reasonable choices for getting a single text vector from a sequence: (1) take the CLS-equivalent token at position 0, (2) mean-pool all token embeddings, (3) take the embedding at the

last (EOS) token. CLIP uses option 3 — the model is causal, so the EOS token has seen the whole sentence.

For our small text encoder, mean-pooling (with masking on padding) is simplest and works well. Try all three later if you're curious — it's a one-line change.

🔑 CURIOSITY CORNER — Projection heads: why a separate space for the alignment?

You might wonder: why have a separate projection head at all? Why not just use the encoder outputs directly? The answer is empirical: projection heads consistently help. SimCLR demonstrated this dramatically — they showed that representations evaluated for downstream tasks are best taken from before the projection, not after.

The intuition: the projection head can specialize for the contrastive objective (which is somewhat artificial — random batch negatives, etc.) while preserving more general features in the encoder. The encoder is what you actually use after training; the projection head is scaffolding.

Toy Alignment Experiment and Writeup

Before training on real data, run one toy alignment experiment to verify your loss actually causes alignment to happen. This is the cheapest possible "does my training pipeline work" check.

The toy experiment in `task5/toy_alignment.py`:

11. Create 32 fixed random vectors of dim 192. Call these your "image features" — pretend they came from your ViT. Call them I_1, ..., I_32.
12. Create 32 different random vectors of dim 192. Call these your "text features" — T_1, ..., T_32. Make sure these are independent of the I vectors.
13. Skip the encoders entirely. Just use two learnable linear projections to map I → P_img(I) and T → P_text(T), both into a 128-dim space.
14. Train the projections with InfoNCE loss. The model has to learn projections that make P_img(I_i) close to P_text(T_i) but far from all other P_text(T_j).
15. Train for 500 steps. Loss should drop from log(32) ≈ 3.47 to near zero.
16. After training, compute the similarity matrix between all P_img(I_i) and P_text(T_j). Verify it has high values on the diagonal and low values off-diagonal.

Why this matters. This isolates your loss from your encoders. If this toy experiment works (and it should), you know your InfoNCE implementation is correct. If Task 6's real training then fails to converge, you know the problem is in the data pipeline or the encoders — not the loss. This kind of *isolation testing* is one of the most useful debugging habits in machine learning.

🚀 APPLIED STRETCH — Build a tiny semantic search engine

Once your toy alignment works, you have, in miniature, the engine that powers semantic search. Try this:

Take 100 sentences from any text dataset (Wikipedia paragraphs, song lyrics, news headlines — your choice). Run them through a pretrained sentence encoder (sentence-transformers/all-MiniLM-L6-v2 is small and free). Store the embeddings. Now write a function that takes a query string, encodes it the same way, and returns the top-5 most similar sentences by cosine similarity. This is exactly how vector databases like Pinecone and Weaviate work under the hood — at planet scale, with sophisticated indexing, but conceptually identical.

Production semantic search systems power Google's results, code search at GitHub, retrieval-augmented generation in ChatGPT, image search in your phone's gallery app, and music recommendations on Spotify. You just built the core mechanism. Deliverable (optional): task5/semantic_search.py with a working demo, plus 50 words on what you'd change to scale it to a million documents.

Final writeup

In `task5/writeup.md`:

17. Explain InfoNCE in your own words. What is the model being asked to do? Why is it different from standard classification?
18. Why is temperature important? What goes wrong if it's too high? Too low? Why does CLIP learn it as a parameter?
19. Why must embeddings be L2-normalized? What would happen if you skipped normalization?
20. In your toy alignment, plot the loss over training steps. Did it drop smoothly or in jumps? What is your final similarity matrix's average diagonal value vs average off-diagonal value?
21. In the contrastive setup, the batch size determines the number of negatives. Why does increasing batch size typically improve contrastive learning? What is the catch?

Deliverables

- `task5/derivation.pdf` with the math derivations.
- `task5/loss.py` — InfoNCE implementation.
- `task5/test_loss.py` — the four sanity tests.
- `task5/clip_model.py` — full CLIP-style architecture.
- `task5/toy_alignment.py` — the toy training experiment.

- `task5/writeup.md`
- *Optional:* `task5/semantic_search.py`

Task 6 — Training on Flickr8k

Goal: Train your CLIP-style VLM on the Flickr8k dataset of real image-caption pairs. Get the loss to drop meaningfully. Produce attention maps that show the model has learned something. Survive your first long training run with all the debugging that comes with it.

Why this task is the longest

Up to now, every task could be debugged in minutes — the model was small enough that a training run took seconds. Task 6 is different. Each training run takes 30 minutes to several hours. When something goes wrong (and it will), the feedback loop is slow. Half of this task is building the patience and the debugging discipline to work in this regime. Wait time is part of the lesson.

Loading and Preprocessing Flickr8k

Flickr8k. 8,000 images, each with 5 human-written captions. Around 1 GB total. Small enough to fit on disk easily, large enough to actually learn from. Download the dataset and captions file from Kaggle or from a Flickr8k mirror. Place under `task6/data/`.

Build a PyTorch Dataset in `task6/dataset.py`:

```
from torch.utils.data import Dataset
from PIL import Image
import torchvision.transforms as T

class Flickr8kDataset(Dataset):
    def __init__(self, image_dir, captions_file, tokenizer,
                 image_size=64, max_text_len=32, split='train'):
        self.image_dir = image_dir
        self.tokenizer = tokenizer
        self.max_text_len = max_text_len

        # Parse captions: list of (image_filename, caption)
        self.pairs = self._load_pairs(captions_file, split)

        if split == 'train':
            self.transform = T.Compose([
                T.Resize((image_size, image_size)),
                T.RandomHorizontalFlip(),
                T.ColorJitter(0.2, 0.2, 0.2),
                T.ToTensor(),
                T.Normalize(mean=[0.485, 0.456, 0.406],
                            std=[0.229, 0.224, 0.225]),  # ImageNet stats
            ])
        else:
            self.transform = T.Compose([
                T.Resize((image_size, image_size)),
```

```
            T.ToTensor(),
            T.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225]),
        ])

    def __len__(self):
        return len(self.pairs)

    def __getitem__(self, idx):
        filename, caption = self.pairs[idx]
        img_path = os.path.join(self.image_dir, filename)
        image = Image.open(img_path).convert('RGB')
        image = self.transform(image)

        # Tokenize and pad/truncate text
        tokens = self.tokenizer.encode(caption)
        tokens = tokens[:self.max_text_len]
        mask = [1] * len(tokens) + [0] * (self.max_text_len - len(tokens))
        tokens = tokens + [0] * (self.max_text_len - len(tokens))

        return {
            'image': image,
            'tokens': torch.tensor(tokens),
            'mask': torch.tensor(mask),
            'caption': caption,  # keep for debugging/visualization
        }
```

Tokenizer choice. For Flickr8k, a simple word-level tokenizer is fine — vocabulary is small (a few thousand words). Build it by collecting all unique words from training captions, lowercasing, and assigning IDs. Reserve token 0 for padding, 1 for unknown words, 2 for start-of-sequence.

Train/val split. Use the standard Karpathy splits: 6,000 train / 1,000 val / 1,000 test. These splits are published; do not invent your own.

> ⚠ **Image size: 64x64 is the right choice for this project**
>
> Real CLIP uses 224x224 or 336x336 images. We use 64x64 because (a) it lets training finish in hours not days on free Colab, and (b) it keeps the number of patches manageable for our small ViT. The quality of learned alignment will be limited by image resolution — captions like "a man in a red shirt" are easier to learn than "a man with a small mole on his left cheek." That is expected.

Day 1 verification:

22. Load the dataset. Print one sample's image shape, token shape, and human-readable caption. Display the image with matplotlib.

23. Wrap in DataLoader (batch_size=128, num_workers=2). Time one full epoch's data iteration (no model). It should take 10-30 seconds; if much longer, your data loading is slow and will bottleneck training.
24. Print 10 samples' captions to verify your text preprocessing didn't corrupt anything — look for unicode issues, weird punctuation, etc.

> **CURIOSITY CORNER — Why 5 captions per image?**
>
> Crowdsourced captioning datasets typically have multiple captions per image because individual humans describe images differently — one says "a dog playing on grass", another says "a brown puppy in a park". Five captions capture this variation. For contrastive training, this gives you 5x the positive pairs for free; just treat each (image, caption) as an independent training example.
>
> At inference time for retrieval evaluation, having 5 ground-truth captions per image is actually how recall metrics get computed: you find the top-K most similar captions for an image, and you count it as a hit if any of the 5 ground-truth captions are in the top-K.

Building the Training Loop

In `task6/train.py`, build a careful, instrumented training loop. This is not a script you write in 30 minutes — every line here will save you hours of debugging.

Essential components:

- Train loop with cosine LR schedule + linear warmup.
- Validation pass every N training steps (don't wait for end of epoch — you want fast feedback).
- Logging: training loss, validation loss, current learning rate, current temperature, gradient norm, time per step. Log to a CSV file.
- Checkpointing: save best model by val loss.
- Resume capability: be able to restart training from a checkpoint if Colab disconnects.
- Gradient clipping at 1.0 — this matters for transformer training.

Hyperparameter starting point:

```
# These are reasonable starting values; expect to tune
config = {
    'batch_size': 128,
    'lr': 5e-4,
    'weight_decay': 0.05,
    'warmup_steps': 500,
    'total_steps': 10000,
    'grad_clip': 1.0,
    'projection_dim': 128,
```

```
    'embed_dim': 192,
    'image_size': 64,
    'patch_size': 8,    # 8x8 patches -> 64 tokens per image
    'max_text_len': 32,
    'vit_depth': 4,
    'text_depth': 4,
    'n_head': 6,
    'dropout': 0.1,
    'val_every': 200,   # validate every 200 train steps
}
```

CURIOSITY CORNER — Gradient clipping — what it is and when it matters

Gradient clipping rescales the gradient if its norm exceeds a threshold. The most common variant: if ||grad|| > 1.0, scale all gradients by 1.0/||grad||. This prevents occasional huge gradients from blowing up the model.

In RNNs it was essential because of exploding gradients through time. In transformers it's less critical but still helps with stability — early in training, when temperature is being learned and weights are random, you can get spike gradients. Clipping at 1.0 costs nothing and prevents some kinds of training divergence.

CURIOSITY CORNER — Mixed-precision training

If you have a modern GPU (T4 included), train in mixed precision — most operations in fp16, accumulators in fp32. This roughly halves memory and speeds up training 2-3x with minimal accuracy loss. In PyTorch: torch.cuda.amp.autocast() around the forward pass, GradScaler for the backward.

For this project it's optional but worth trying. Search: "PyTorch automatic mixed precision tutorial."

The First Training Run and Diagnostics

Run training for 10,000 steps. Watch the logs.

What healthy training looks like:

- Loss starts at approximately `log(batch_size)` = log(128) ≈ 4.85.
- Loss drops rapidly in the first 500-1000 steps to around 3.5-4.0.
- Loss then continues to decrease slowly. By step 10000, expect ~2.5-3.0 on training, slightly higher on validation.
- Learned temperature starts at 0.07 and typically drifts to around 0.01-0.03 during training.
- Gradient norm spikes occasionally but stays bounded (this is what clipping is for).

What unhealthy training looks like, and what to do:

Symptom 1 — Loss not decreasing at all

- Check that gradients reach all parameters (run a dummy backward + assert param.grad is not None for every param).
- Check that L2 normalization is happening before the loss.
- Check that labels in cross_entropy are torch.arange(N) and not something else.
- Try a smaller learning rate (1e-4 instead of 5e-4).

Symptom 2 — Loss drops then plateaus way too high (e.g. at 4.5)

- Almost certainly a temperature problem. Print learned temperature; if it's stuck at initialization, your log-temperature gradient is being clipped or zeroed.
- Try a higher learning rate (1e-3).
- Check batch size — too small (under 64) makes contrastive learning weak because there are too few negatives.

Symptom 3 — Loss is NaN

- Almost always a numerical-stability issue. Check that you're using log-temperature (not raw temperature).
- Check that you clamp log-temperature.
- Lower learning rate. Reduce gradient clip threshold.
- If using mixed precision, this is a common issue — try disabling it to isolate.

Symptom 4 — Train loss decreases, val loss stays flat or increases

- Classic overfitting. Increase dropout to 0.2 or 0.3.
- Add more data augmentation on images.
- Reduce model size.
- Train for fewer steps.

> ⚠ **Don't tune blindly. Form a hypothesis first.**
>
> The most common failure mode at this stage is panic-tuning — changing 5 things at once because something looks off. Every change you make should be motivated by a specific hypothesis about what is wrong. "Loss is plateauing at 4.5, I think temperature is stuck, let me print it before changing

> anything" is the right loop. "Loss looks bad, let me try lr=1e-5 and dropout=0.5 and batch_size=64 all at once" is how you waste two days.
>
> Keep a debugging log: hypothesis → change → observation → next step. Future you will thank you.

Building Retrieval Evaluation

Training loss alone doesn't tell you if alignment is working. You need a downstream evaluation. The standard for CLIP-style models is *retrieval*: given a text query, find the matching image among 1000 candidates (image retrieval); given an image, find the matching caption among 5000 candidates (text retrieval). Compute Recall@K for K = 1, 5, 10.

Build `task6/eval.py`:

```
@torch.no_grad()
def evaluate_retrieval(model, val_loader):
    model.eval()
    all_image_embeds = []
    all_text_embeds = []
    all_captions = []
    all_image_ids = []

    for batch in val_loader:
        img_e = model.encode_image(batch['image'].cuda())
        txt_e = model.encode_text(batch['tokens'].cuda(),
                                  batch['mask'].cuda())
        all_image_embeds.append(F.normalize(img_e, dim=-1).cpu())
        all_text_embeds.append(F.normalize(txt_e, dim=-1).cpu())
        all_captions.extend(batch['caption'])
        all_image_ids.extend(batch['image_id'])

    image_embeds = torch.cat(all_image_embeds, 0)  # (N_img, D)
    text_embeds  = torch.cat(all_text_embeds,  0)  # (N_txt, D)

    # Image-to-text retrieval: for each image, rank all captions
    similarity = image_embeds @ text_embeds.T       # (N_img, N_txt)

    # ... compute Recall@1, @5, @10 over the diagonal-ish structure
    # (remember: each image has 5 matching captions in val)
```

Recall@K for Flickr8k: given an image, compute its similarity to all captions in the validation set. Get the top-K captions. The image scores a hit if any of its 5 ground-truth captions appears in the top-K. Average across all images.

Expected results for a well-trained small model on Flickr8k at 64x64:

- Image → Text Recall@1: 15-25%

- Image → Text Recall@5: 40-55%
- Image → Text Recall@10: 55-70%
- Text → Image numbers slightly lower.

Random-chance Recall@1 for 1000 candidates is 0.1%. Anything significantly above chance means your model has learned something.

> 🔑 **CURIOSITY CORNER — Recall@K beyond retrieval**
>
> Recall@K is everywhere in information retrieval and recommendation systems. "Of the K items I showed the user, did the right answer appear?" Search engines optimize Recall@10. Recommendation systems track Recall@20 or @50. Vector databases for RAG systems measure Recall@K of their nearest-neighbor approximation versus exact search.
>
> Once you've computed it for image-text retrieval, you can compute it for any embedding-based system. It's a remarkably general metric.

Attention Maps and Qualitative Analysis

Time for the payoff. With a trained model, redo the attention map visualization from Task 4 Day 4. Pick 5-10 images from validation. For each, run through your trained model and visualize how patches contribute to the matching caption's similarity.

One way: similarity map. For an image and its caption, compute the dot product between each *patch's* projected embedding and the caption's projected embedding. Normalize and overlay as a heatmap. Patches with high similarity to the caption should correspond to the image regions the caption describes.

Save your best 5 examples as `task6/qualitative.png`. Pick examples where you can see localization clearly. Also pick 1-2 failure cases — images where the model got the wrong caption or attended to the wrong region. These are often more informative than the successes.

Beyond retrieval — try zero-shot classification:

25. Take a small set of class names: "a photo of a dog", "a photo of a car", "a photo of a person", etc.
26. Encode each as text through your text encoder. You now have one embedding per class.
27. Take an image. Encode it. Compute similarity to each class embedding.
28. Predicted class = argmax similarity.
29. Try this on a few Flickr8k images. Does it work? When?

This is exactly zero-shot CLIP — no labels, no fine-tuning, just "which class name's embedding is closest to the image embedding?" The fact that this works at all on a model trained only on Flickr8k is part of the magic.

Applied Stretch Tasks (Choose One or More)

By now you have a trained CLIP-style model and the machinery to use its embeddings. This day is dedicated to actually applying these to small real-world demos. Pick at least one of the following; ambitious mentees can attempt two.

🚀 APPLIED STRETCH — Application 1 — Image Captioning Demo (uses cross-attention from Task 4)

You trained a CLIP-style model with no cross-attention. But your Task 4 cross-attention machinery is still in your codebase. Take your trained ViT encoder, freeze it, and attach a small decoder-style text transformer with cross-attention. Train this on Flickr8k with simple next-token-prediction loss (cross-entropy on the next caption word) for 2000 steps.

The result: an image → caption model. Feed in a new image, generate a caption token-by-token using greedy or top-k sampling. This is the architecture behind early image captioning systems. Production systems (BLIP, Flamingo) are much larger and use much more data, but the architecture is the same.

Deliverable: task6/captioning_demo.py with 10 generated captions for 10 validation images, plus the human ground-truth caption for comparison. Don't expect Pulitzer-winning prose — "a dog on grass" is a perfectly good result for our scale.

🚀 APPLIED STRETCH — Application 2 — Text-to-Image Search Engine

Precompute embeddings for all 8000 Flickr8k images using your trained model. Store them in a single tensor (N_img, D).

Build a tiny command-line app: prompt the user for a query ("a person riding a bicycle", "sunset over water", "two dogs playing"). Encode the query through your text encoder. Compute similarity to all image embeddings. Return the top-5 image filenames. Display them in a grid.

This is, conceptually, the entire algorithm behind Google Images (with CLIP-style models), the search bar in your phone's photo app, and the visual search in e-commerce sites. Production versions use much larger models, GPU-accelerated nearest-neighbor search (FAISS), and serve billions of images — but the core algorithm fits in 50 lines.

Deliverable: task6/text_to_image_search.py + a screenshot of 3 example queries with their top-5 results.

APPLIED STRETCH — Application 3 — Zero-Shot Image Classification on a Custom Set

Pick a domain. Five animal species. Eight weather conditions. Twelve types of food. Whatever interests you. Write out the class names as natural-language descriptions: "a photo of a husky dog", "a photo of a poodle", etc.

Find 20-30 test images for these categories (Google image search downloads are fine for personal exploration). Run zero-shot classification on them using your trained model. Compute accuracy.

Then try "prompt engineering" — for each class, try multiple templates: "a photo of a husky dog", "an image of a husky", "a husky in the snow". Use the average of the three text embeddings for each class. Does accuracy improve? This is a real technique used in deployed CLIP systems.

Deliverable: task6/zero_shot.py with a confusion matrix and a paragraph on what worked and what didn't.

APPLIED STRETCH — Application 4 — Duplicate Image Detector

A very practical application: detecting near-duplicate images in a large collection. Take Flickr8k. For each image, find its 5 most similar images by image-embedding cosine similarity. Inspect a few — you'll find genuine near-duplicates (same scene from a slightly different angle), images of the same subject, and images with similar composition.

This is used in production for: detecting copyright infringement (a Reddit upload that's a re-hosted Instagram image), photo deduplication in cloud storage, finding similar listings on marketplace apps, and content moderation (flagging known harmful images by similarity to a reference set).

Deliverable: task6/duplicate_finder.py with 5 example query images and their 5 nearest neighbors displayed in a grid.

APPLIED STRETCH — Application 5 — Cross-Modal Retrieval Beyond Vision-Language

Contrastive learning is modality-agnostic. The same loss works for any two paired views. As a tiny demo of this generality, take a small set of audio-caption pairs (e.g., from AudioCaps, or just labeled environmental sounds), use a pretrained audio encoder (e.g., from torchaudio) plus your text encoder, and train a small alignment head with InfoNCE.

This is more ambitious — handling audio data and integrating a pretrained encoder is non-trivial — but it demonstrates that contrastive alignment is a recipe, not a vision-specific technique. The same machinery powers CLAP (audio-language), CodeBERT (code-language), and protein sequence-structure models in biology.

This is the most advanced of the stretch tasks. Only attempt if everything else feels easy.

Final Writeup and Cleanup

In `task6/writeup.md`, a longer writeup than previous tasks since this is the culmination of the technical work in the project:

30. Report your training curve. At what step did loss stop improving meaningfully? What was your final train and val loss?
31. Report your retrieval metrics — Recall@1, @5, @10 in both directions. Where is your model strongest? Weakest?
32. Describe the bugs you hit during training and how you debugged them. This is genuinely the most valuable section for your future self and for other people who read your writeup.
33. Show 3-5 qualitative examples of attention maps. Which ones surprised you?
34. From your zero-shot classification experiment: which classes worked well? Which failed? Speculate on why — was it lack of training data with those concepts, ambiguity of the class names, or something else?
35. Which applied stretch task did you attempt? Show the deliverable. What did you learn that you didn't learn from training the core model?
36. What is one ablation or experiment you'd run next if you had another week? (Don't write "more data and bigger model" — that's the boring answer. Be specific: what hyperparameter, what architectural variant, what data treatment.)

Repo cleanup:

- Your README should explain the project, your final results, how to reproduce them.
- Code should be organized: separate dataset, model, loss, training, eval, viz.
- A single requirements.txt with pinned versions.
- A weights file (your best checkpoint) — small enough to upload to GitHub, or hosted via Google Drive with a download script.

Deliverables

- `task6/dataset.py`
- `task6/train.py`
- `task6/eval.py`
- `task6/qualitative.png` — attention maps for 5+ examples.
- `task6/training_curve.png` — loss vs steps for train and val.
- `task6/best_model.pt` — trained weights.

- `task6/writeup.md`
- *At least one Applied Stretch deliverable.*

What Comes After Task 6

Tasks 5 and 6 are the technical climax of the project. If you have a working trained model with reasonable retrieval numbers, you have built — from scratch, with your own hands — the same kind of system that powers modern search, recommendation, and multimodal AI products. Everything that follows is polish.

- **Final report and presentation.** Write up the project as a coherent narrative. Build slides. Practice the talk. The technical work is done; this is about communicating it well.

If Task 5 was learning the rule that makes alignment happen, Task 6 was watching alignment actually happen on data you can see. The Applied Stretches were the bridge from this project to the world. Everything you built here is the same recipe that powers half of modern AI products — at much larger scale, with much more data, but with the same core ideas you now hold in your hands.